Research Notes on DBSCAN

DBSCAN

>> Section 1

Reachability is not a symmetric relation: by definition, only core points can reach non-core points. The opposite is not true, so a non-core point may be reachable, but nothing can be reached from it. Therefore, a further notion of connectedness is needed to formally define the extent of the clusters found by DBSCAN. Two points p and q are density-connected if there is a point o such that both p and q are reachable from o. Density-connectedness is symmetric. A cluster then satisfies two properties:

>> Section 2

Extensions Generalized DBSCAN (GDBSCAN) is a generalization by the same authors to arbitrary "neighborhood" and "dense" predicates. The ε and minPts parameters are removed from the original algorithm and moved to the predicates. For example, on polygon data, the "neighborhood" could be any intersecting polygon, whereas the density predicate uses the polygon areas instead of just the object count. Various extensions to the DBSCAN algorithm have been proposed, including methods for parallelization, parameter estimation, and support for uncertain data. The basic idea has been extended to hierarchical clustering by the OPTICS algorithm. DBSCAN is also used as part of subspace clustering algorithms like PreDeCon and SUBCLU. HDBSCAN* is a hierarchical version of DBSCAN which is also faster than OPTICS, from which a flat partition consisting of the most prominent clusters can be extracted from the hierarchy.

>> Section 3

Optimization Criterion DBSCAN optimizes the following loss function: For any possible clustering C = { C 1 , … , C l } {\displaystyle C=\{C_{1},\ldots ,C_{l}\}} out of the set of all clusterings C {\displaystyle {\mathcal {C}}} , it minimizes the number of clusters under the condition that every pair of points in a cluster is density-reachable, which corresponds to the original two properties "maximality" and "connectivity" of a cluster:

>> Section 4

Availability Different implementations of the same algorithm were found to exhibit enormous performance differences, with the fastest on a test data set finishing in 1.4 seconds, the slowest taking 13803 seconds. The differences can be attributed to implementation quality, language and compiler differences, and the use of indexes for acceleration.

>> Section 5

Preliminary Consider a set of points in some space to be clustered. Let ε be a parameter specifying the radius of a neighborhood with respect to some point. For the purpose of DBSCAN clustering, the points are classified as core points, (directly-) reachable points and outliers, as follows:

>> Section 6

Relationship to spectral clustering A spectral implementation of DBSCAN is related to spectral clustering in the trivial case of determining connected graph components — the optimal clusters with no edges cut. However, it can be computationally intensive, up to O (n 3) {\displaystyle O(n^{3})} . Additionally, one has to choose the number of eigenvectors to compute. For performance reasons, the original DBSCAN algorithm remains preferable to its spectral implementation.

>> Section 7

Apache Commons Math contains a Java implementation of the algorithm running in quadratic time. ELKI offers an implementation of DBSCAN as well as GDBSCAN and other variants. This implementation can use various index structures for sub-quadratic runtime and supports arbitrary distance functions and arbitrary data types, but it may be outperformed by low-level optimized (and specialized) implementations on small data sets. mlpack includes an implementation of DBSCAN accelerated with dual-tree range search techniques. PostGIS includes ST_ClusterDBSCAN – a 2D implementation of DBSCAN that uses R-tree index. Any geometry type is supported, e.g. Point, LineString, Polygon, etc. R contains implementations of DBSCAN in the packages dbscan and fpc. Both packages support arbitrary distance functions via distance matrices. The package fpc does not have index support (and thus has quadratic runtime and memory complexity) and is rather slow due to the R interpreter. The package dbscan provides a fast C++ implementation using k-d trees (for Euclidean distance only) and also includes implementations of DBSCAN*, HDBSCAN*, OPTICS, OPTICSXi, and other related methods. scikit-learn includes a Python implementation of DBSCAN for arbitrary Minkowski metrics, which can be accelerated using k-d trees and ball trees but which uses worst-case quadratic memory. A contribution to scikit-learn provides an implementation of the HDBSCAN* algorithm. pyclustering library includes a Python and C++ implementation of DBSCAN for Euclidean distance only as well as OPTICS algorithm. SPMF includes an implementation of the DBSCAN algorithm with k-d tree support for Euclidean distance only. Weka contains (as an optional package in latest versions) a basic implementation of DBSCAN that runs in quadratic time and linear memory. linfa includes an implementation of the DBSCAN for the rust programming

language. Julia includes an implementation of DBSCAN in the Julia Statistics's Clustering.jl package.

>> Section 8

MinPts: As a rule of thumb, a minimum minPts can be derived from the number of dimensions D in the data set, as minPts ≥ D + 1. The low value of minPts = 1 does not make sense, as then every point is a core point by definition. With minPts ≤ 2, the result will be the same as of hierarchical clustering with the single link metric, with the dendrogram cut at height ε. Therefore, minPts must be chosen at least 3. However, larger values are usually better for data sets with noise and will yield more significant clusters. As a rule of thumb, minPts = 2·dim can be used, but it may be necessary to choose larger values for very large data, for noisy data or for data that contains many duplicates. ε: The value for ε can then be chosen by using a k-distance graph, plotting the distance to the k = minPts-1 nearest neighbor ordered from the largest to the smallest value. Good values of ε are where this plot shows an "elbow": if ε is chosen much too small, a large part of the data will not be clustered; whereas for a too high value of ε, clusters will merge and the majority of objects will be in the same cluster. In general, small values of ε are preferable, and as a rule of thumb only a small fraction of points should be within this distance of each other. Alternatively, an OPTICS plot can be used to choose ε, but then the OPTICS algorithm itself can be used to cluster the data. Distance function: The choice of distance function is tightly coupled to the choice of ε, and has a major impact on the results. In general, it will be necessary to first identify a reasonable measure of similarity for the data set, before the parameter ε can be chosen. There is no estimation for this parameter, but the distance functions needs to be chosen appropriately for the data set. For example, on geographic data, the great-circle distance is often a good choice. OPTICS can be seen as a generalization of DBSCAN that replaces the ε parameter with a maximum value that mostly affects performance. MinPts then essentially becomes the minimum cluster size to find. While the algorithm is much easier to parameterize than DBSCAN, the results are a bit more difficult to use, as it will usually produce a hierarchical clustering instead of the simple data partitioning that DBSCAN produces. Recently, one of the original authors of DBSCAN has revisited DBSCAN and OPTICS, and published a refined version of hierarchical DBSCAN (HDBSCAN*), which no longer has the notion of border points. Instead, only the core points form the cluster.